0150. Evaluate Reverse Polish Notation

Medium · Stack

leetcode.com/problems/evaluate-reverse-polish-notation

Problem

- Evaluate a postfix expression given as tokens (+, -, , / and integers). Division truncates toward zero.
- Example: ["2","1","+","3",""] → 9.

Intuition

- In postfix, an operator applies to the two most recent values, so a stack of operands is all we need.

Brute Force

- Repeatedly find the first operator and collapse it with its two predecessors: O(n^2).

Optimal Approach

- Number: push it.
- Operator: b = pop, a = pop (order matters!), push a op b.
- Result is the single remaining value.

Worked Example

```
["4","13","5","/","+"].

4       [4]
13      [4,13]
5       [4,13,5]
/       13/5=2    [4,2]
+       4+2=6     [6]
```

Java Solution

```java
public class EvaluateReversePolishNotation {
    public int evalRPN(String[] tokens) {
        Deque<Integer> st = new ArrayDeque<>();
        for (String t : tokens) {
            switch (t) {
                case "+" -> st.push(st.pop() + st.pop());
                case "*" -> st.push(st.pop() * st.pop());
                case "-" -> { int b = st.pop(), a = st.pop(); st.push(a - b); }
                case "/" -> { int b = st.pop(), a = st.pop(); st.push(a / b); }
                default -> st.push(Integer.parseInt(t));
            }
        }
        return st.pop();
    }
}
```

Complexity

- Time: O(n). Space: O(n).

Pitfalls

- Operand order for - and /: the second pop is the left operand.
- "-3" is a number, not an operator; compare the whole token.